

SIMSHIELD PROJECT GUIDE

SIMShield: Mobile Banking Fraud Protection

A clear guide on how we stop SIM swaps, eSIM theft, and account takeovers

PROJECT TYPE	PLATFORMS	PROTECTION FOCUS	TEST STATUS
Full-Stack Security	Android App & Node.js	SIM Swap / ATO	26/26 Tests Passed

1 Problem Understanding

When people use online banking, UPI, or mobile wallets, their phone number is tied to everything. Banks use SMS OTPs to verify logins, password resets, and money transfers.

The core issue is that phone numbers were never built to act as secure passwords:

- SIM Swapping:** A scammer goes to a telecom store, pretends to be the victim, and gets a new SIM card for the victim's number.
- eSIM Hijacking:** Attackers hack into carrier self-service accounts and move the victim's eSIM to their own phone in seconds.
- The SMS OTP Trap:** Once scammers get the phone number, they receive all incoming banking OTPs. They can reset the banking password, add a new beneficiary account, and drain the money before the victim even knows their phone lost network.
- Why phone apps alone cannot stop this:** A banking app on a hacker's phone can easily lie about what is happening if the phone is rooted. All real fraud checks have to happen on the bank's secure server, not on the phone.

The Real Challenge: Regular users change SIM cards and upgrade phones all the time. We cannot simply lock people out every time a SIM changes. We need to look at what happens next (like a new device login and password reset) to catch the bad guys without annoying real customers.

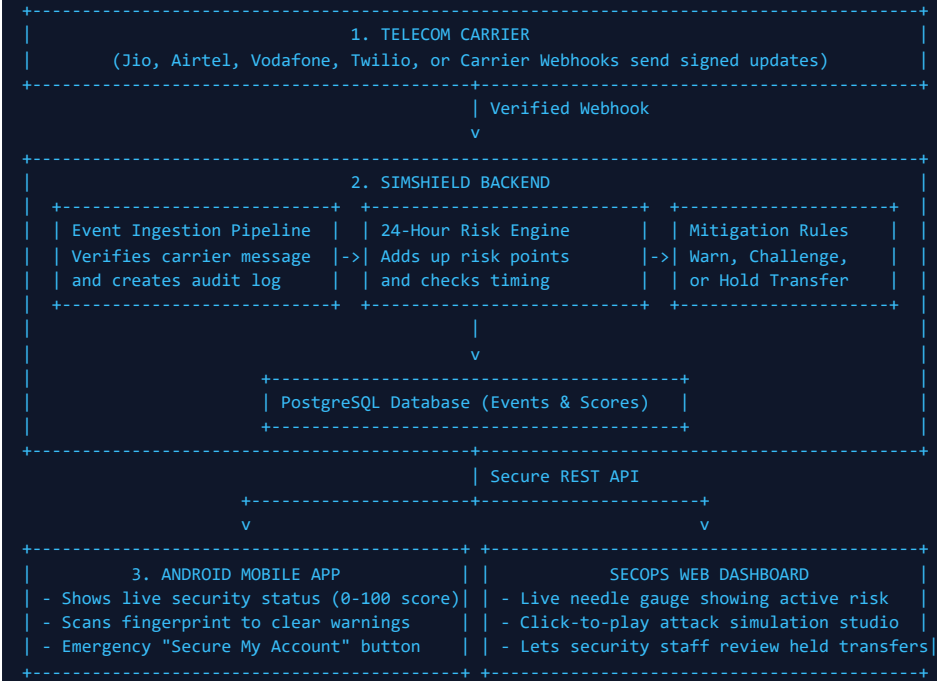
2 Proposed Solution

SIMShield stops these attacks by connecting the bank directly with telecom carriers. Here is how it works:

- The Server Makes the Decisions:** Telecom partners notify our backend the moment a SIM or eSIM is swapped. The phone app never decides its own safety score.
- 24-Hour Timeline Check:** We look at user actions over a 24-hour window. If someone swaps a SIM and then immediately tries to add a beneficiary and transfer money from an unknown device, we recognize the attack pattern and block the transfer.
- Real Fingerprint Verification:** Instead of sending an SMS OTP (which the scammer would receive), we ask the user to verify with their real physical fingerprint on their trusted phone.
- Panic Button ("Secure My Account"):** If a user spots suspicious activity, one tap logs out all other devices, freezes wire transfers, and stops all SMS logins instantly.

3 System Architecture

The platform is split into three simple parts working together:



4 Key Features

A. Smart Risk Scoring

Every suspicious action adds risk points. If the total goes up, protections kick in:

Event	Points	What It Means	Reason Code
SIM Swap / eSIM Change	+30 pts	SIM card or eSIM was replaced	RECENT_SIM_CHANGE
Number Ported	+25 pts	Phone number moved to another company	NUMBER_PORTED
New Device Sign-In	+20 pts	Logged in from an unfamiliar phone	NEW_DEVICE
Password / PIN Reset	+15 pts	Password reset right after SIM change	PASSWORD_RESET_AFTER_SIM_CHANGE
New Beneficiary Added	+15 pts	New payee added to receive funds	NEW_BENEFICIARY_AFTER_SIM_CHANGE
Suspicious High Transfer	+20 pts	Sudden large money transfer attempt	ABNORMAL_TRANSACTION

B. Action Levels

- 0 to 29 (LOW) : Normal. Everything works smoothly with zero friction.
- 30 to 49 (MEDIUM) : Warning banner shown on the app. The account stays open.
- 50 to 79 (HIGH) : Asks for fingerprint verification before allowing sensitive actions.
- 80 to 100 (CRITICAL) : Suspicious transfers are placed on hold and flagged for investigation.

C. Mobile App & Admin Tools

- Physical Fingerprint Scanner:** Uses Android's built-in fingerprint reader to verify the real user.
- "Secure My Account" Button:** An emergency freeze button that kicks out hackers and halts transfers.
- Admin Dashboard:** A web page at localhost:3000 where security staff can watch attacks in real time and manage cases.
- Demo Sandbox:** 8 buttons in the mobile app so anyone can test SIM swaps and takeovers during demonstrations.

5 Technology Stack

Layer	Technology	Why We Chose It
Backend Server	Node.js (ES Modules)	Super fast, handles lots of events at once, and doesn't need heavy third-party packages.
Database	PostgreSQL	Rock-solid database to store events, user scores, and security reports safely.
Mobile App	Android (Java, SDK 26-35)	Uses native Android tools for fingerprint scanning and system notifications.
Security	HMAC-SHA256 Hashing	Hashes phone numbers and device IDs so private customer data is never stored in plain text.
Admin Dashboard	HTML5, CSS3, Vanilla JS	Clean, lightweight web screen that runs in any browser without complicated setup.
Automation & Testing	Docker & GitHub Actions	Builds the app automatically and runs 26 automated tests on every code update.

6 Implementation Plan

We built and tested the system step by step:

1. **Step 1 (Core Engine):** Built the backend risk calculator with the 24-hour sliding window.

2. **Step 2 (Database Setup):** Created PostgreSQL tables to store events, scores, and fraud reports.

3. **Step 3 (API Routes):** Built secure API endpoints with signature checks so carriers can send updates safely.

4. **Step 4 (Mobile App):** Added real hardware fingerprint scanning and the emergency lock button to the Android app.

5. **Step 5 (Admin Dashboard):** Built the live web dashboard with the attack simulation player.

6. **Step 6 (Testing & CI):** Configured GitHub Actions to test the backend and compile the Android APK automatically.

7 Test Results

What We Tested	Expected Result	Result
Single SIM Swap	Gives score 30 (Warning only, does not lock user)	Passed
Full Takeover Attack	SIM Swap + New Device + Password Reset = Score 100 (Transfer Held)	Passed
24-Hour Expiration	Events older than 24 hours are dropped from the score	Passed
Fingerprint Reader	Only a matched fingerprint clears the security alert	Passed
Automated Test Suite	All unit and integration tests run cleanly	26 / 26 Passed